

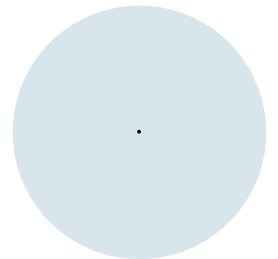
Einführung in die Programmierung (WIN+BIT)

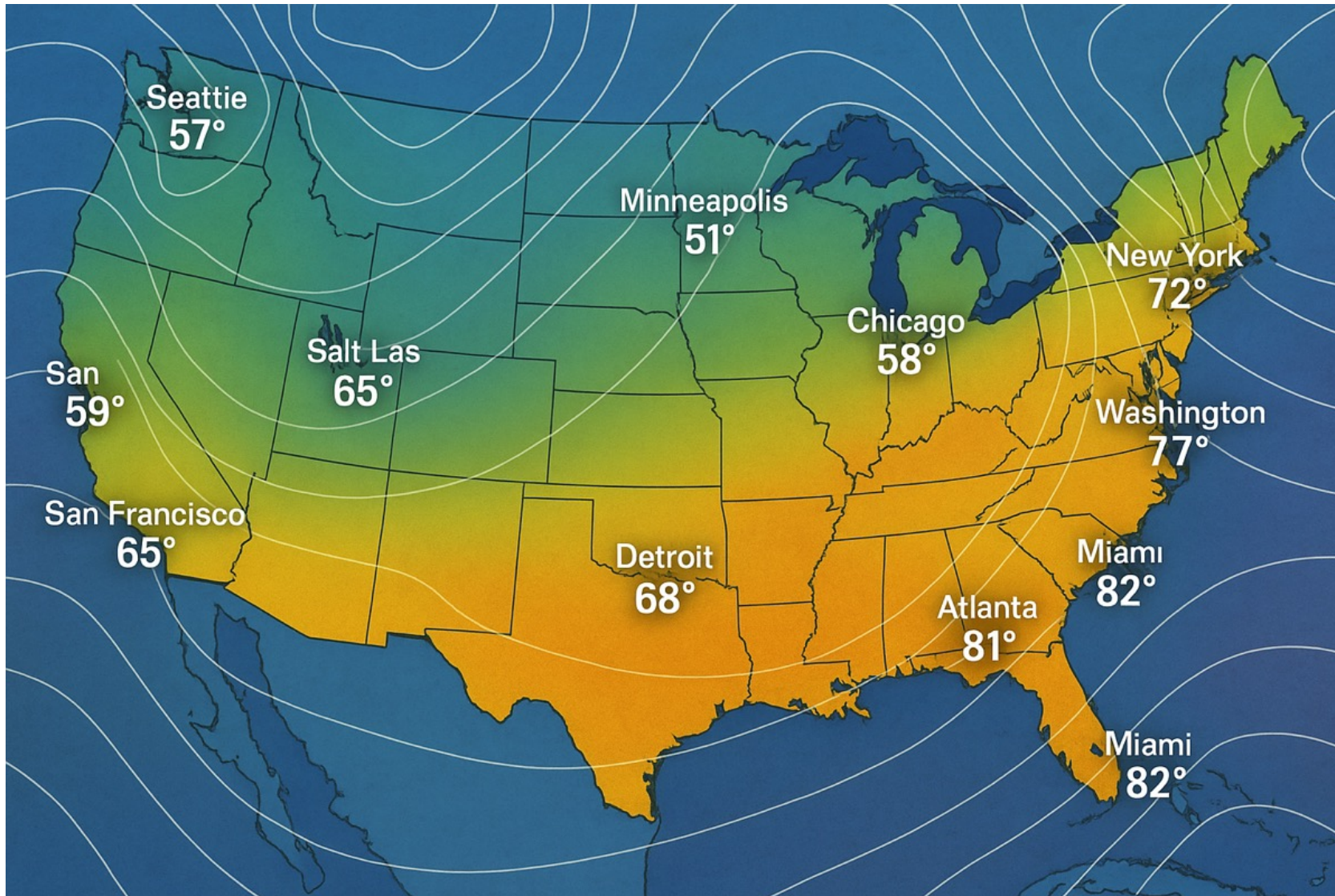
Klassenmethoden, Prozedurale Programmierung

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Motivation





Generiert mit ChatGPT 5

Motivation

Auf US Wetterkarten sind Temperaturen in Fahrenheit angegeben. Wir wollen diese in Celsius umrechnen:

```
void main() {  
    int detroitF = 68;  
    int chicagoF = 58;  
    int atlantaF = 81;  
  
    double detroitC = (detroitF - 32) * 5.0 / 9.0;  
    double chicagoC = (chicagoF - 32) * 5.0 / 9.0;  
    double atlantaC = (atlantaF - 32) * 5.0 / 9.0;  
}
```

Der Code macht hier **dreimal** (fast) dasselbe.

Kann man das nicht einfacher hinschreiben?



code07.
Fahrenheit
tSimple

– Jetzt ist

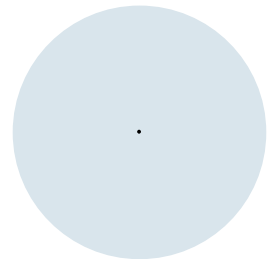
```
void main() {  
    int detroitF = 68;  
    int chicagoF = 58;  
    int atlantaF = 81;  
    double detroitC = fahrenheitToCelsius(detroitF);  
    double chicagoC = fahrenheitToCelsius(chicagoF);  
    double atlantaC = fahrenheitToCelsius(atlantaF);  
}
```

```
double fahrenheitToCelsius(double fahrenheit) {  
    double result = (fahrenheit - 32) * 5.0 / 9.0;  
    return result;  
}
```



code07.
Fahrenheit
Method

Methoden definieren



Methode

- Eine **Methode** ist ein **Block von Anweisungen**, der eine **bestimmte Aufgabe** ausführt und (optional) ein **Ergebnis zurückgibt**.
- Einfache Beispiele:
 - Fahrenheit in Celsius umrechnen
 - Seemeilen in Kilometer umrechnen
 - Lineare Abschreibung berechnen
- Folgendes muss dabei definiert werden:
 - Was sind die **Eingaben**?
 - Welche **Verarbeitung** soll durchgeführt werden?
 - Was ist die **Ausgabe**?

Temperaturumwandlung

- Eine **Methode** ist ein **Block von Anweisungen**, der eine **bestimmte Aufgabe** ausführt und (optional) ein **Ergebnis** zurückgibt.

- **Aufgabe:** Umrechnung von von **Fahrenheit** in **Celsius**

- **Eingabe:**

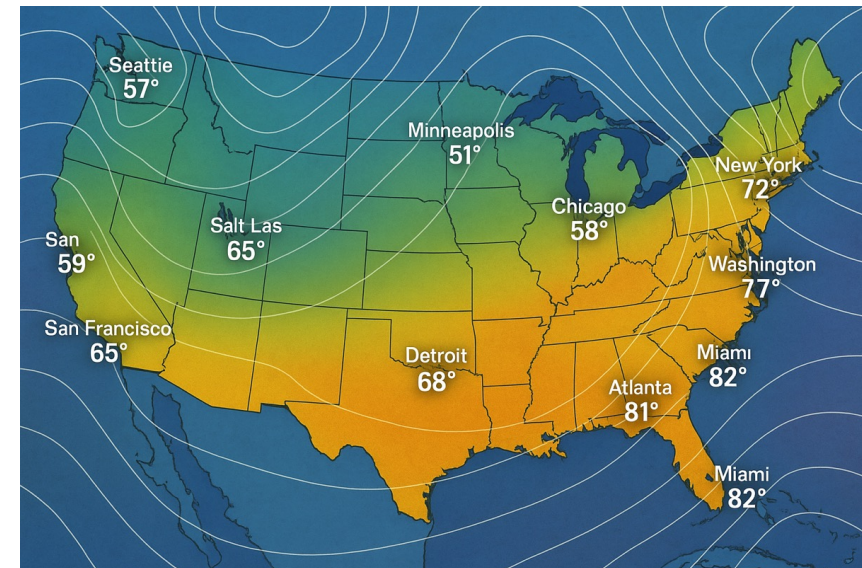
T_F

- **Verarbeitung:**

$$T_C = (T_F - 32) \times \frac{5}{9}$$

- **Ausgabe:**

T_C



Methoden

Methodendefinition:

```
double fahrenheitToCelsius(double fahrenheit) {  
    double result = (fahrenheit - 32) * 5.0 / 9.0;  
    return result;  
}
```

Methodenaufruf:

```
int detroitF = 68;  
int chicagoF = 58;  
int atlantaF = 81;  
  
double detroitC = fahrenheitToCelsius(detroitF);  
double chicagoC = fahrenheitToCelsius(chicagoF);  
double atlantaC = fahrenheitToCelsius(atlantaF);
```



code07.

Fahrenheit
Method

Methoden definieren

Stück Programmcode, dem ein **Name** gegeben wird, unter dem es aufgerufen werden kann

Kann **Parameter** (Argumente) enthalten, die beim Aufruf durch Werte ersetzt werden

Kann ein Ergebnis **zurückliefern**

- Deklaration einer Methode
`public static <Rückgabety> <Methodenname>(<FormaleParameter>) {`
 ...
`}`
- <FormaleParameter> als komma-getrennte Liste "<Typ> <Name>, ..."

Methodenbeispiel

Stück Programmcode, dem ein **Name** gegeben wird, unter dem es aufgerufen werden kann

Kann **Parameter** (Argumente) enthalten, die beim Aufruf durch Werte ersetzt werden

Kann ein Ergebnis **zurückliefern**

- Deklaration einer Methode

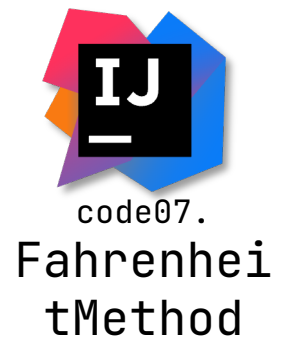
```
public static int sum(int a, int b) {  
    return a + b;  
}
```

- <FormaleParameter> als komma-getrennte Liste "<Typ> <Name>, ..."

Rückgabewerte

- Mit `return <Wert>` wird eine Methode beendet und ein Wert zurückgegeben
- Datentyp von `<Wert>` muss zuweisungskompatibel mit Datentyp der Methode sein (vgl. Deklaration)
 - im Beispiel: ein `double`-Wert wird berechnet – kompatibel mit Rückgabebetyp `double`

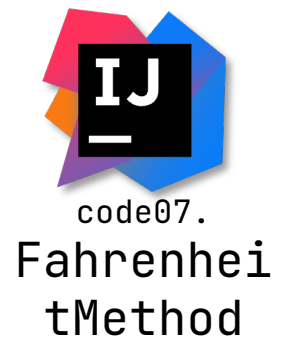
```
double fahrenheitToCelsius(double fahrenheit) {  
    double result = (fahrenheit - 32) * 5.0 / 9.0;  
    return result;  
}
```



Rückgabewerte

- Mit `return <Wert>` wird eine Methode beendet und ein Wert zurückgegeben
- Datentyp von `<Wert>` muss zuweisungskompatibel mit Datentyp der Methode sein (vgl. Deklaration)
 - im Beispiel: ein `double`-Wert wird berechnet – kompatibel mit Rückgabebetyp `double`

```
double fahrenheitToCelsius(double fahrenheit) {  
    double result = (fahrenheit - 32) * 5.0 / 9.0;  
    return result;  
}
```



Beispiel

- Implementiere ein Methode, welche die Jährliche Lineare Abschreibung einer Investition berechnet.
- Eingaben:

AK

Anschaffungskosten

RW

Restwert (am Ende der Nutzungsdauer)

ND

Nutzungsdauer (in Jahren)

- Berechnung: $A = \frac{AK - RW}{ND}$

- Ausgabe: A



Beispiel

– Implementiere ein Methode, welche die Jährliche Lineare Abschreibung einer Investition berechnet.

– Eingaben:

AK

Anschaffungskosten

RW

Restwert (am Ende der Nutzungsdauer)

ND

Nutzungsdauer (in Jahren)

– Berechnung: $A = \frac{AK - RW}{ND}$

– Ausgabe: A

```
double lineareAbschreibung(double anschaffungskosten, double restwert, double  
nutzungsdauer) {  
    return (anschaffungskosten - restwert) / nutzungsdauer;  
}
```



Rückgabewert void

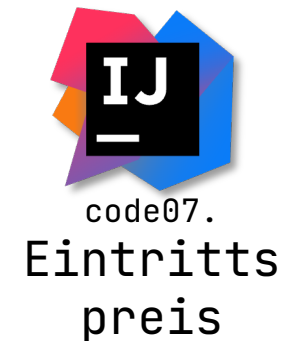
- void ist engl. für "Leerstelle", "unbesetzt"
- Datentyp void steht für "kein Rückgabewert"; return wird dann ohne <Wert> aufgerufen und **darf auch fehlen**

```
void printFahrenheitToCelsius(int fahrenheit, String city) {  
    double celsius = (fahrenheit - 32) * 5.0 / 9.0;  
    System.out.printf("%s: = %d F / %.1f C%n", city, fahrenheit, celsius);  
}
```


Beispiel: Eintrittspreis

Methodendefinition:

```
double berechneEintrittspreis(int wochentag) {  
    double result = -1;  
    switch (wochentag) {  
        case 0 -> result = 4;  
        case 1, 2 -> result = 5;  
        case 3 -> result = 6;  
        case 4 -> result = 4;  
        case 5 -> result = 8;  
        case 6 -> result = 10;  
    }  
    return result;  
}
```



Beispiel: Eintrittspreis

Methodenaufruf:

```
void main() {
    boolean eingabe = true;
    Scanner scanner = new Scanner(System.in);
    while (eingabe) {
        System.out.print("Wochentag: ");
        int wochentag = scanner.nextInt();
        double entrittspreis = berechneEintrittspreis(wochentag);
        System.out.printf("Eintrittspreis: %.2f\n", entrittspreis);
        System.out.print("Weiterer Eingabe J/N? ");
        String weiter = scanner.next();
        eingabe = weiter.toLowerCase().equals("j");
    }
}
```

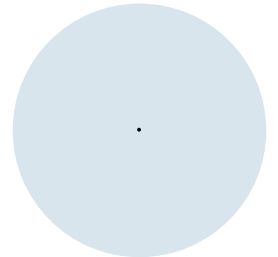


code07.
Eintritts
preis

Ab jetzt: Methoden verwenden!

- Verwenden Sie Methoden mit aussagekräftigen Namen, um Ihren Code besser zu strukturieren
- Beispiele aus den Übungsaufgaben
 - `double handelsspanne(double verkaufspreis, double einkaufspreis)`

Methoden überladen



Methodensignatur

- Die **Methodensignatur** umfasst den **Namen der Methode** und die **Typen der Parameter** (in ihrer Reihenfolge, s. [Oracle](#)) – nicht jedoch die *Namen* der Parameter und den Rückgabety

```
public <Rückgabety> <Methodenname>(<FormaleParameter>) {  
    // ...  
}
```

Beispiele:

- double **max**(double a, double b)
- double **max**(double d1, double d2)
- int **max**(double a, double b)



gleiche Signatur trotz unterschiedlicher Parameternamen!

gleiche Signatur trotz unterschiedlicher Rückgabetypen!

Überladen einer Methode

Überladen von Methodennamen:

- **Gleicher Methodenname** für **unterschiedliche Signaturen** möglich, Auswahl zur Laufzeit aufgrund der **Datentypen** der **tatsächlichen** Parameter
(Achtung: engl. *actual* = dt. *tatsächlich* und nicht *aktuell*)

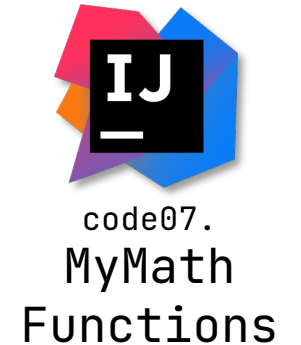
```
void println() { ... }  
void println(String s) { ... }  
void println(double d) { ... }  
void println(String s, String t) { ... }  
String println(String s) { ... }
```

nicht möglich, da es die gleiche Signatur wie `void println(String s)` hat und zur Laufzeit nicht davon unterschieden werden kann.

Überladen einer Methode, weiteres Beispiel

- min-Methode ist überladen: Einmal für int, einmal für double-Parameter

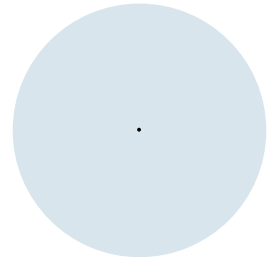
```
public class MyMathFunctions {  
    public static int min(int a, int b) {  
        return a <= b ? a : b;  
    }  
  
    public static double min(double a, double b) {  
        return a <= b ? a : b;  
    }  
}
```



- Welche Methode aufgerufen wird, entscheidet die Laufzeitumgebung anhand der Parametertypen:

```
min(1, 5); // erste Methode  
min(27.5, 13.8); // zweite Methode
```

Verwendung von Methoden anderer Klassen



Methoden in derselben Klasse

```
void main() {  
    ...  
    double betrag = berechneGesamtPfand(eingabe);  
    ...  
}  
  
double berechneGesamtPfand(String eingabe) {  
    double betrag = 0.0;  
    for (int index = 0; index < eingabe.length(); index++) {  
        char c = eingabe.charAt(index);  
        double pfand = berechnePfand(c);  
        betrag += betrag+pfand;  
    }  
    return betrag;  
}  
  
double berechnePfand(char c) {  
    ..  
}
```



code07.
Pfand
Methode

main-Methode

- Die main-Methode ist auch eine **Klassenmethode**, damit sie automatisch von der Laufzeitumgebung gestartet werden kann.
- Wenn Ihre main-Methode lang wird, können Sie den Code in mehrere **Klassenmethoden** auslagern*:

```
void main(args) {  
    int n = readInt("Bitte Zahl >= 0 eingeben: ");  
    printSquareNumbers(0, n);  
}  
  
int readInt(String message) { ... }  
void printSquareNumbers(int from, int to) {  
    ...  
}
```

Übersicht der Schritte
abstrakte Beschreibung

Detaillierte Umsetzung
konkrete Ausführung

*noch besser ist es, wenn Sie den Code mit OOD-Prinzipien umbauen, s. spätere VL.

Standardmethoden

- Sind Teil der Java Standard Bibliothek
- Implementieren allgemeine Funktionalität die oft benötigt wird
- Sind auf verschiedene **Klassen** aufgeteilt um Übersichtlichkeit zu gewährleisten

```
double square = Math.pow(5, 2);  
double random = Math.random();  
String zahlAlsString = String.valueOf(2.355);
```

- Ganz zu schweigen von diesem bekannter Methodenaufruf:

```
System.out.println("Hello World!");
```



code07.
MathExample

Beispiele aus dem SDK

– Math.abs(...)

static double **abs**(double a)

Returns the absolute value of a double value.

static float **abs**(float a)

Returns the absolute value of a float value.

static int **abs**(int a)

Returns the absolute value of an int value.

static long **abs**(long a)

Returns the absolute value of a long value.

– Math.max(...)

static double **max**(double a, double b)

Returns the greater of two double values.

static float **max**(float a, float b)

Returns the greater of two float values.

static int **max**(int a, int b)

Returns the greater of two int values.

static long **max**(long a, long b)

Returns the greater of two long values.

– Math.min(...)

static double **min**(double a, double b)

Returns the smaller of two double values.

static float **min**(float a, float b)

Returns the smaller of two float values.

static int **min**(int a, int b)

Returns the smaller of two int values.

static long **min**(long a, long b)

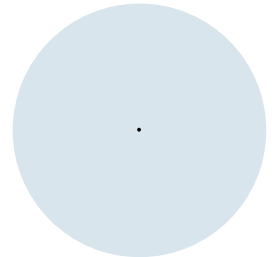
Returns the smaller of two long values.

Beispiele aus dem SDK

`System.out.println(...)`

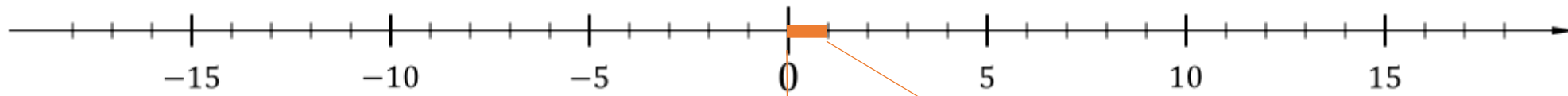
void	<code>println()</code>	Terminates the current line by writing the line separator string.
void	<code>println(boolean x)</code>	Prints a boolean and then terminate the line.
void	<code>println(char x)</code>	Prints a character and then terminate the line.
void	<code>println(char[] x)</code>	Prints an array of characters and then terminate the line.
void	<code>println(double x)</code>	Prints a double and then terminate the line.
void	<code>println(float x)</code>	Prints a float and then terminate the line.
void	<code>println(int x)</code>	Prints an integer and then terminate the line.
void	<code>println(long x)</code>	Prints a long and then terminate the line.
void	<code>println(Object x)</code>	Prints an Object and then terminate the line.
void	<code>println(String x)</code>	Prints a String and then terminate the line.

Zufallszahlen



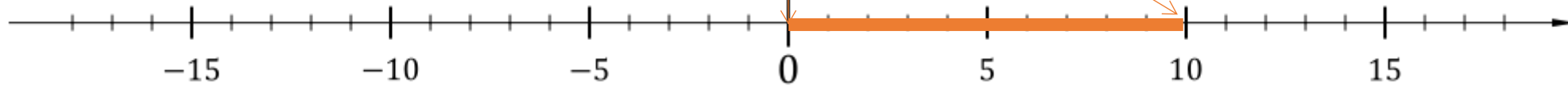
Zufallszahlen

- `Math.random()` liefert Zufallszahlen im Intervall $[0..1)$

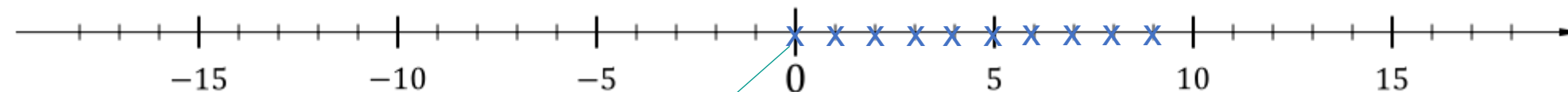


Für andere Intervalle geeignet multiplizieren, z.B.

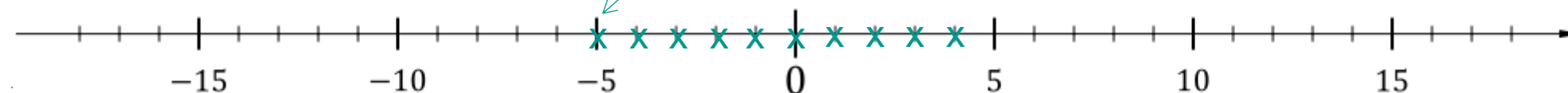
- `Math.random() * 10` → $[0..10)$



- `(int) (Math.random() * 10)` → Ganzzahlen $0..9$



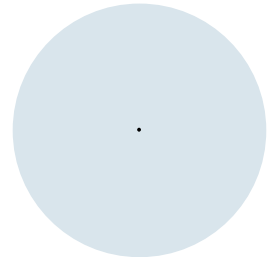
- `((int) (Math.random() * 10)) - 5` → Ganzzahlen $-5..+4$



Zufallszahlen

- Alternative Implementierungen verfügbar (schneller, gleichmäßiger verteilt)
 - [java.util.Random](#)
 - [java.security.SecureRandom](#)
- ➔ für Übungsaufgaben und Klausur **nicht nötig**
- Hintergrundwissen: Der Zufallsgenerator von Java ist ein sog. Pseudozufallszahlengenerator.
- [Wikipedia](#): Pseudozufallszahlengeneratoren erzeugen eine Zahlenfolge, die zwar zufällig aussieht, es aber nicht ist, da sie durch einen deterministischen Algorithmus berechnet wird. Solche Pseudozufallszahlen sind von Computern wesentlich einfacher zu erzeugen und in praktisch allen höheren Programmiersprachen verfügbar. Bei jedem Start der Berechnung mit gleichem Startwert (engl. seed, d. h. Saatkorn) wird die gleiche Folge erzeugt, weshalb diese deterministisch erzeugten Pseudozufallszahlen bei hinreichend genauer Dokumentation später reproduziert werden können.
- Der Startwert ist meistens die aktuelle Zeitspanne in Millisekunden, die seit dem 1.1.1970 vergangen ist.

Methodentypen



Klassenmethoden

- Klassenmethoden anderer Klassen kann man über den Namen der Klasse direkt aufrufen.
 - Math ist eine Klasse mit vielen Klassenmethoden

```
double randomValue = Math.random() * 10;  
double twoToThePowerOfTen = Math.pow(2.0, 10.0);  
double minimum = Math.min(11.0, 42.0);
```

- Klassenmethoden nennt man auch **statische Methoden**. Wir werden später noch andere Methodentypen kennen lernen.

Instanzmethoden

- In Java gibt es Methoden, die man erst aufrufen kann, wenn man eine Instanz einer Klasse erzeugt hat
 - new erzeugt eine Instanz der Klasse Scanner
 - Hier kann man jetzt die Methode nextLine aufrufen

```
Scanner inputScanner = new Scanner(System.in);  
String input = inputScanner.nextLine();
```

- Das String-Literal "Hallo" erzeugt eine String-Instanz
 - Auf dieser Instanz kann man jetzt die Methode toUpperCase aufrufen

```
String hallo = "Hallo Welt";  
String halloUpper = hallo.toUpperCase();
```

- Wie man eigene Klassen schreibt und Instanzen erzeugt, lernen wir **später**
- Dies ist die sogenannte *Objektorientierte Programmierung* (Klassen und Objekte)

Auszug aus der String API

jshell>



StringMethods1

Return Type	Method	Description
int	length()	Returns the length of this string.
boolean	isEmpty()	Returns true if, and only if, length() is 0! Only true for the empty string "".
char	charAt(int index)	Returns the char value at the specified index.
String	substring(int beginIndex)	Returns a string that is a substring of this string, starting from beginIndex. Example: "abc".substring(1) is "bc".
String	substring(int beginIndex, int endIndex)	Returns a string that is a substring of this string. endIndex is not included. Example: "abc".substring(1,2) is "b".
boolean	equals(String s)	Compares this string to the specified String. Example: "abc".equals("abc") is true.

"GIRAFFEN"
01234567

Zeichen Nr. 3 ist A

Achtung: String der Länge 8,
aber Nummerierung beginnt bei 0

Strings nie mit == vergleichen,
sondern immer über .equals!!!

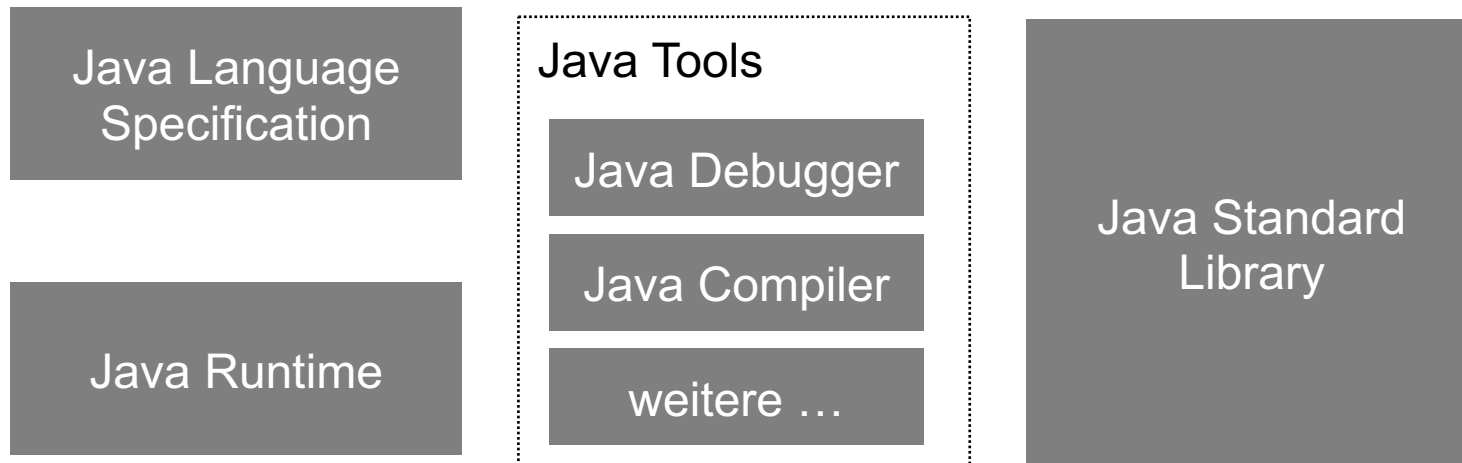


StringCompare

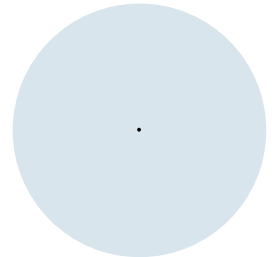
Die Programmiersprache Java und das JDK, was gehört dazu?

Java Development Kit

- Java Language Specification
- Java Tools
- Java Runtime (Java Virtual Machine)
- Java Standardbibliothek (Standard Library)



Zusammenfassung: Prozedurales Programmieren



Prozedurales Programmieren

- Die **prozedurale Programmierung** ergänzt das imperative Konzept aufeinander folgender Befehle um den Ansatz, ein Programm in überschaubare Teile zu zerlegen. Je nach Programmiersprache werden diese Teile Unterprogramm, Routine, Prozedur, Methode oder Funktion genannt. Es lassen sich **Parameter übergeben** und **Ergebnisse zurückgeben**. (nach [Wikipedia](#))
- Zerlegung dient verschiedenen Zwecken:
 - Code-Duplikation vermeiden (wie im Beispiel mit dem Vergleich der Durchschnittsnoten und weiteren Beispielen)
 - Verschiedene Abstraktionsebenen
 - Grundstruktur (high-level) in der main-Methode
 - Detaillierte Funktionalität innerhalb der Untermethoden
 - Programm lässt sich so deutlich besser lesen
 - siehe Beispiel nächste Folie

Methoden: Wiederverwendbarkeit

- Methoden vermeiden Duplizierung von gleichem oder ähnlichem Code

```
drawTriangle(3, 3, 3, 1, 1, 3);  
drawTriangle(6, 6, 8, 6, 7, 9);
```

- versus

```
drawLine(3, 3, 3, 1);  
drawLine(3, 1, 1, 3);  
drawLine(1, 3, 3, 3);  
  
drawLine(6, 6, 8, 6);  
drawLine(8, 6, 7, 9);  
drawLine(7, 9, 6, 6);
```


Methoden: Besser lesbaren Code

- Methoden machen den Code besser lesbar (selbst, wenn man die Methode nur genau einmal verwendet!):

```
double abschlussnote = durchschnitt(muendlicheNote, schriftlicheNote);
```

- versus

```
// Durchschnittsberechnung  
double abschlussnote = (muendlicheNote + schriftlicheNote) / 2.0;
```

- der Kommentar im zweiten Beispiel kann beim ersten Beispiel entfallen
- Durchschnitt ist noch einfach zu durchschauen, aber es gibt auch kompliziertere Berechnungen